

Question 2

Step 1:

In the merge_sort function we are recursively dividing the array arr into two halves at each step with the line "mid = (low + high) // 2", this division recursion occurs until all subarrays each have a size of 1. The number of times the array can be divided is equivalent to the log of the array then $\log(n)$ where n is the size of the original array.

Step 2:

The merge function sorts the two subarrays back into the original array, in the worst case every element in the array will need to be compared and copied back into the original array. For the first part of the function, the two subarrays are traversed and compared to sort into the original array, and then once one array has been completely copied the function moves on to the while loop that corresponds to the array that has not finished traversing. Then the remaining elements in said array will be copied into the original array. The worst case time complexity for the function merge will be $O(n)$ where every element in each subarray must be sorted.

Step 3:

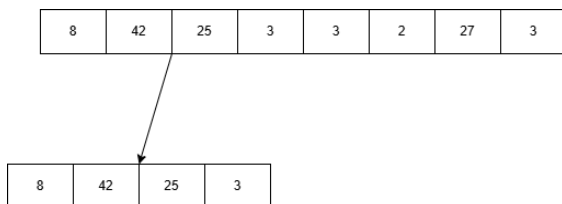
Combining both steps we reach a total time complexity of $O(\log(n)) * O(n) = O(n\log(n))$

Question 3

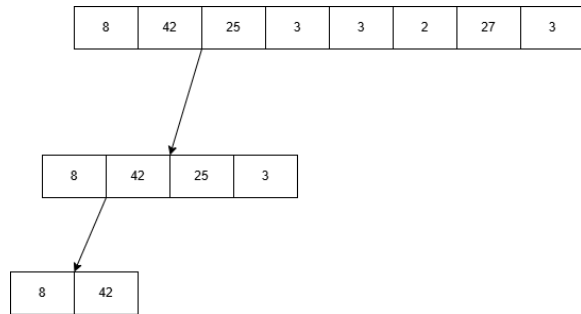
Step 0: Call merge sort with initial array

8	42	25	3	3	2	27	3
---	----	----	---	---	---	----	---

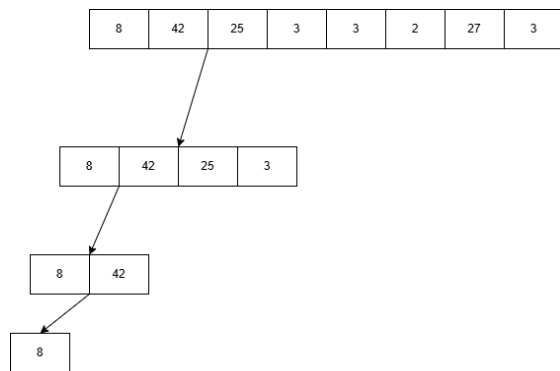
Step 1: Divide the left half of the array into a subarray



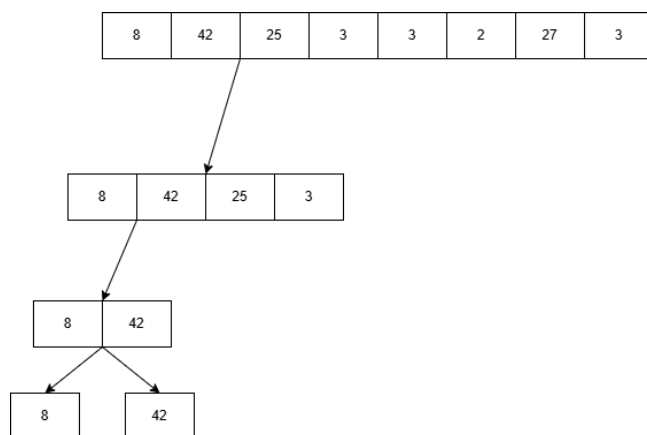
Step 2: Divide the left half of the subarray into a subarray



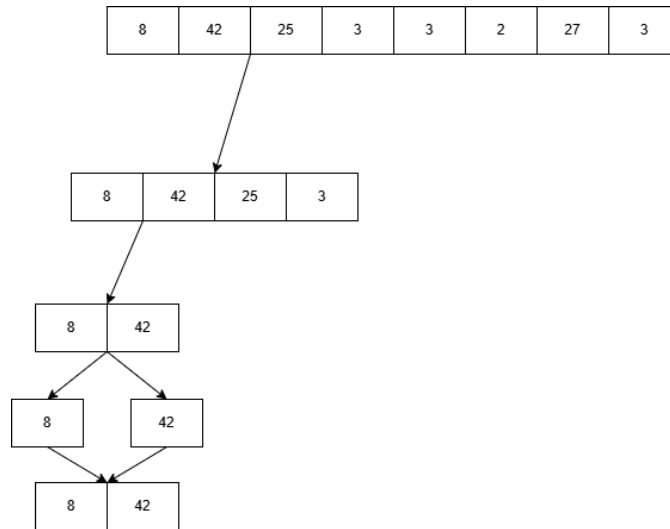
Step 3: Divide the left half of the subarray into a subarray



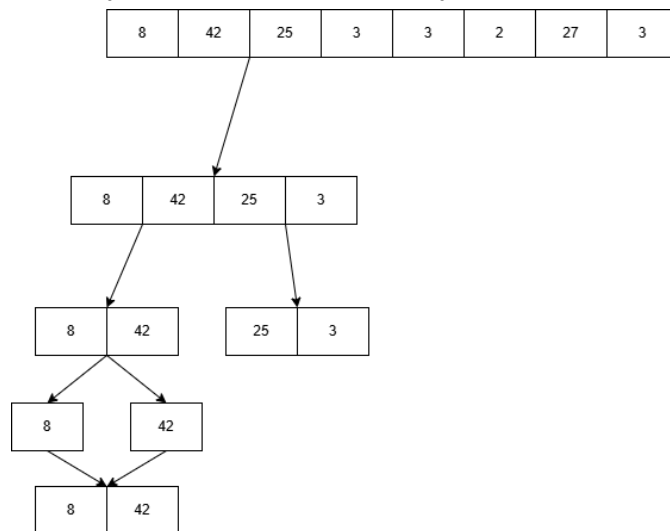
Step 5: Since low is equal to high, the recursive function returns and the right side of the subarray is divided into a subarray.



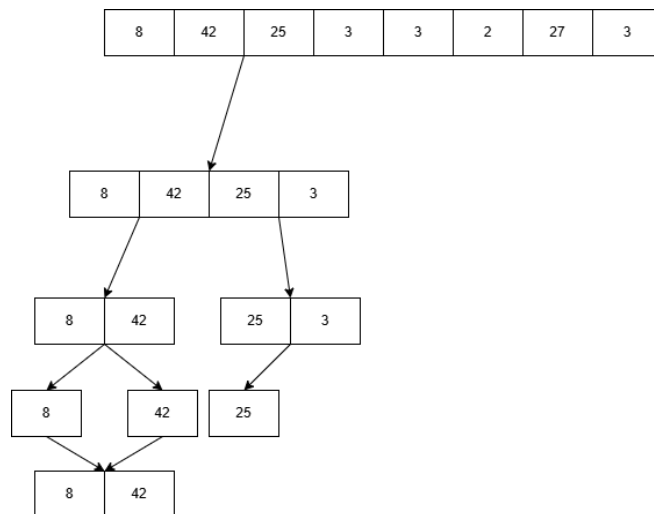
Step 6: Compare the values within the subarrays and merge them into the original array in a smallest to greatest sorted order.



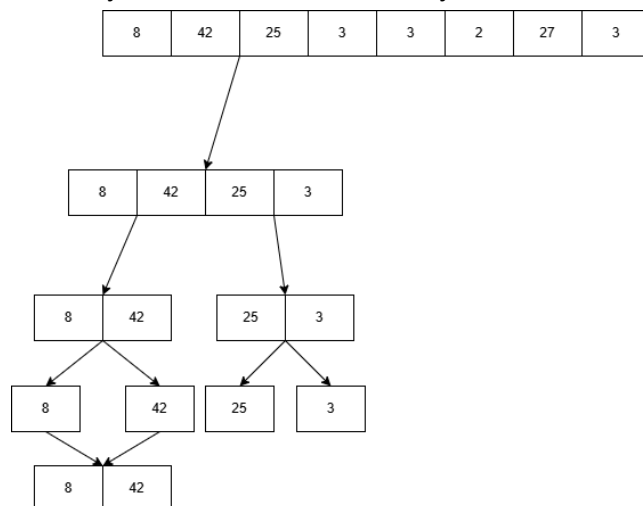
Step 7: Since low is equal to high, the recursive function returns and the right side of the subarray is divided into a subarray.



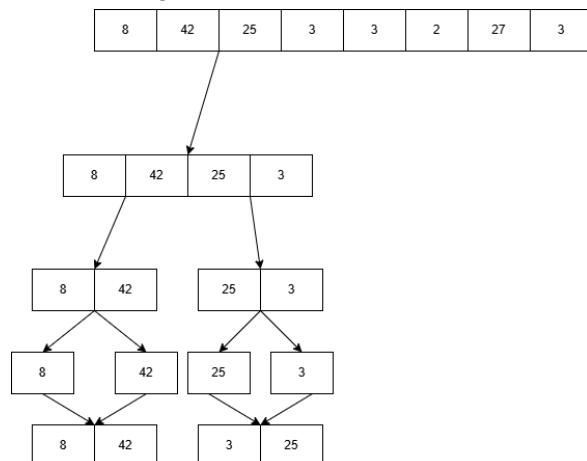
Step 8: Divide the left half of the right subarray into a subarray



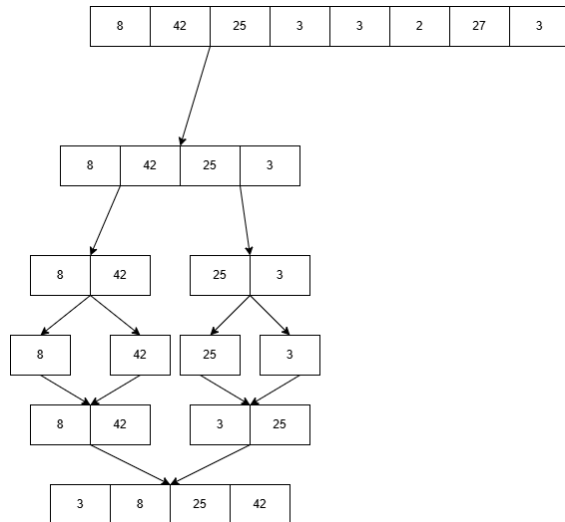
Step 9: Since low is equal to high, the recursive function returns and the right side of the subarray is divided into a subarray.



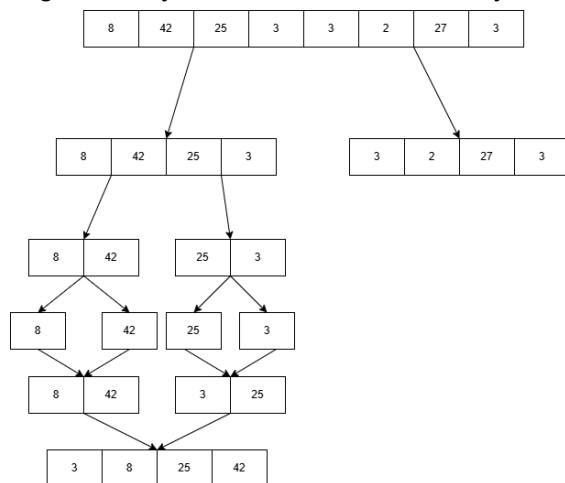
Step 10: Compare the values within the subarrays and merge them into another subarray in a smallest to greatest sorted order.



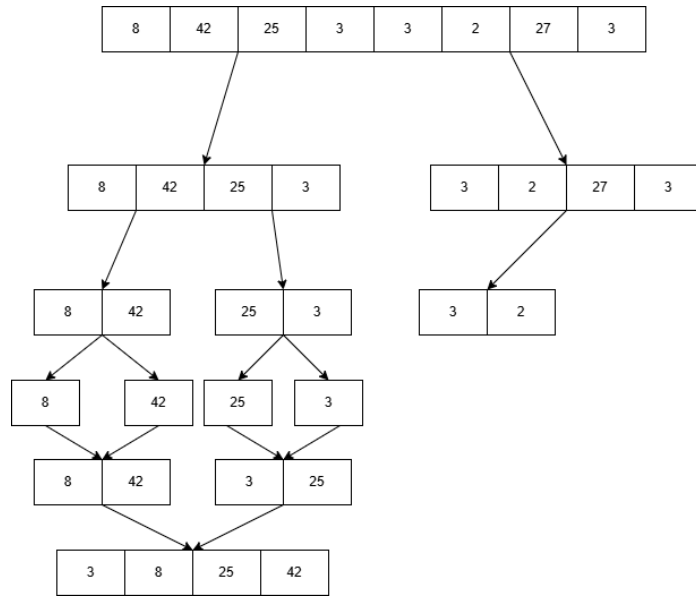
Step 11: Compare the values within the two merged subarrays and merge them into another subarray in a smallest to greatest sorted order.



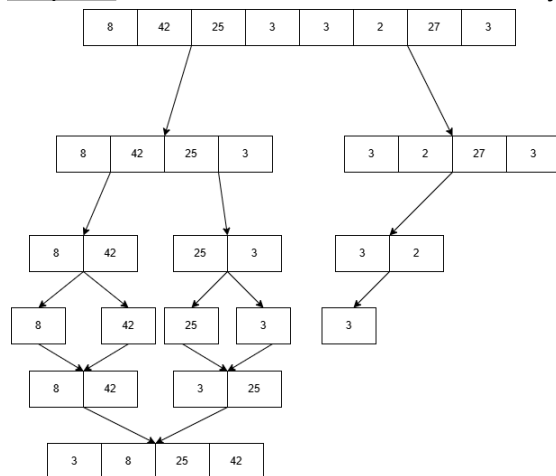
Step 12: Since low is equal to high, the recursive function returns and the right side of the original array is divided into a subarray.



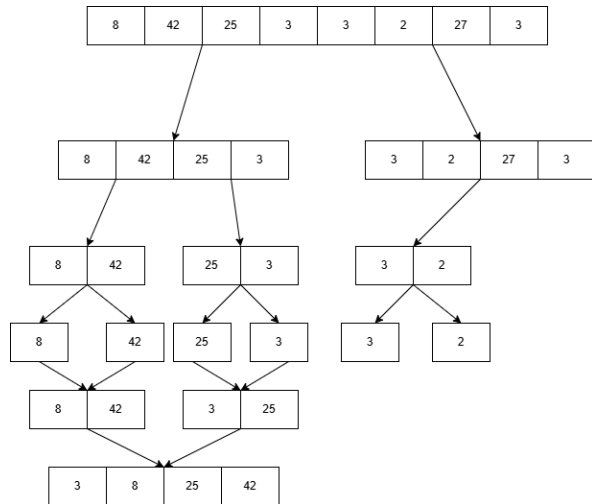
Step 13: Divide the left half of the subarray into a subarray



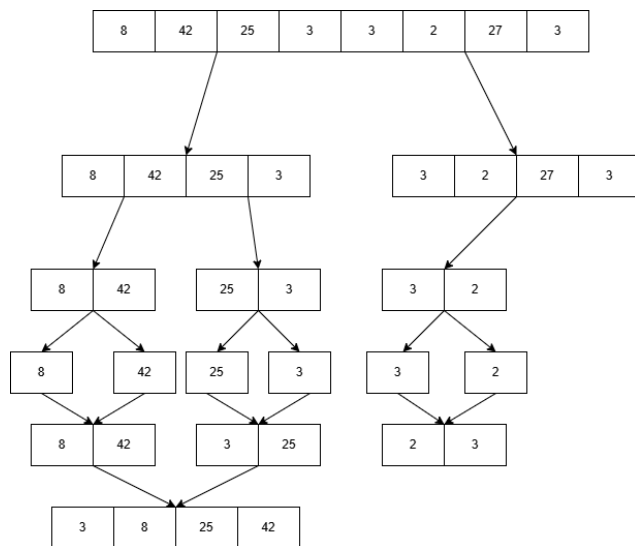
Step 14: Divide the left half of the subarray into a subarray



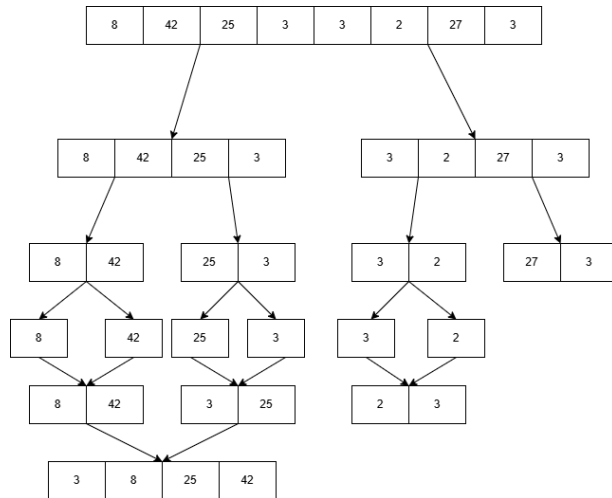
Step 15: Since low is equal to high, the recursive function returns and the right side of the subarray is divided into a subarray.



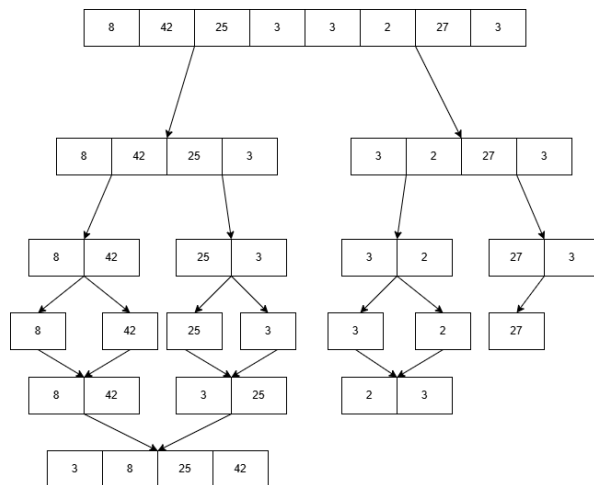
Step 16: Compare the values within the subarrays and merge them into another subarray in a smallest to greatest sorted order.



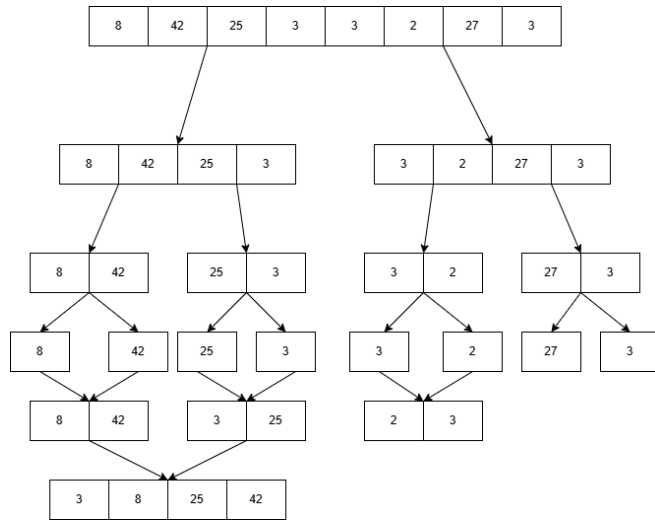
Step 17: Since low is equal to high, the recursive function returns and the right side of the subarray is divided into a subarray.



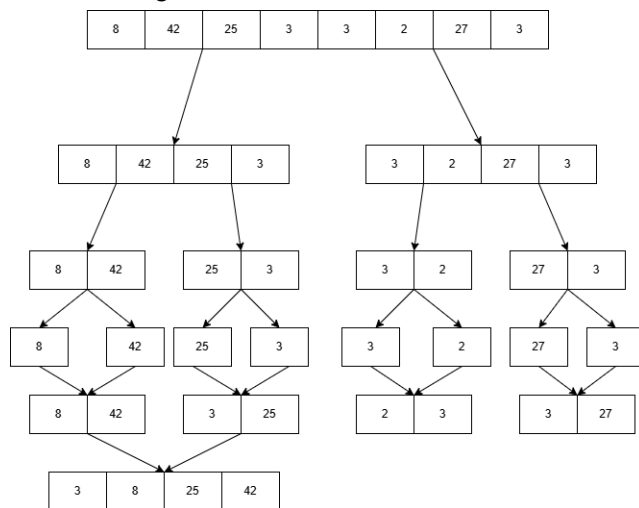
Step 18: Divide the left half of the subarray into a subarray



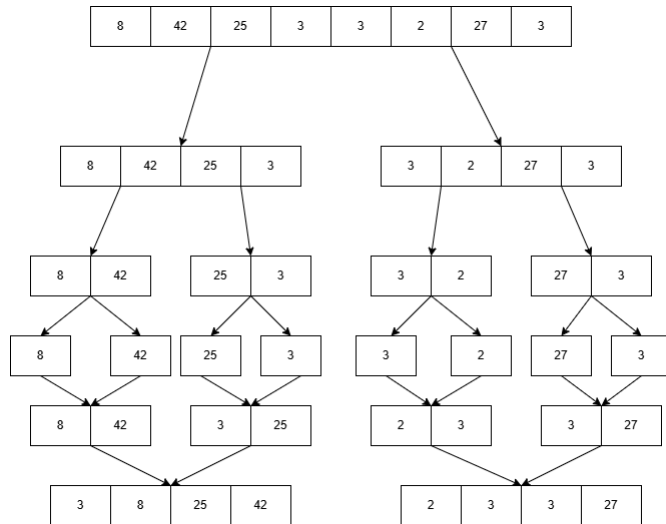
Step 19: Since low is equal to high, the recursive function returns and the right side of the subarray is divided into a subarray.



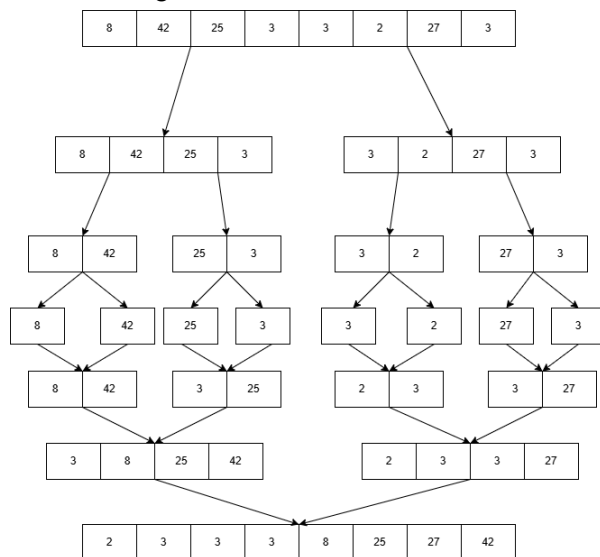
Step 20: Compare the values within the subarrays and merge them into another subarray in a smallest to greatest sorted order.



Step 21: Compare the values within the subarrays and merge them into another subarray in a smallest to greatest sorted order.



Step 22: Compare the values within the subarrays and merge them into another subarray in a smallest to greatest sorted order.



Step 23: Return the sorted array

2	3	3	3	8	25	27	42
---	---	---	---	---	----	----	----

Question 4

The above array has $n=8$ elements, so its time complexity is $(8) * \log_2(8) = (8) * (3) = 24$. This is consistent with the number of steps taken above (0-23). We use $\log_2(n)$ because in our merge sort implementation we are dividing the array in halves, however since $\log_2(n)$ consists of a constant it is simplified to $\log(n)$ in our complexity analysis.